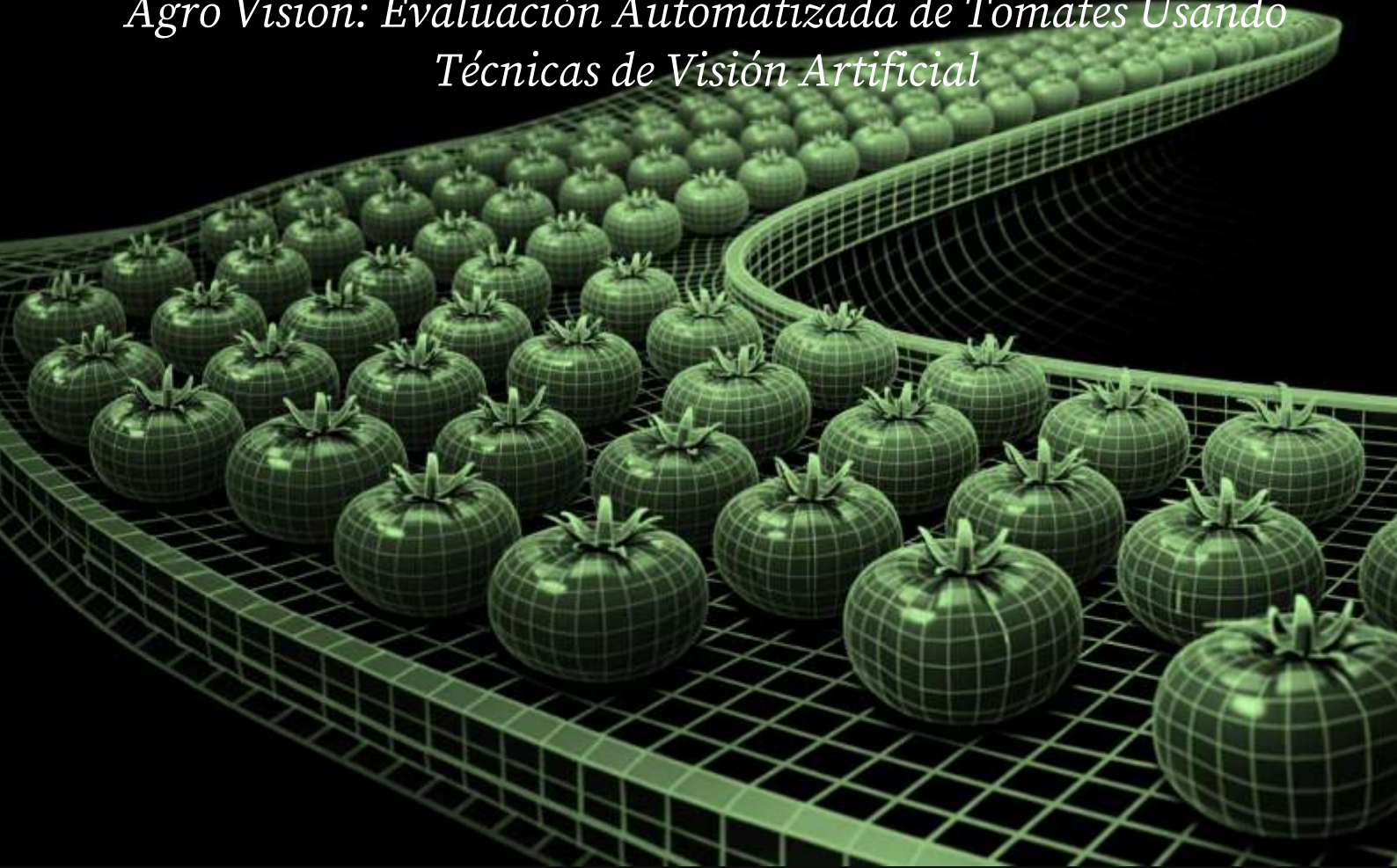




Universidad Autónoma del Estado de México

Documentación

Agro Vision: Evaluación Automatizada de Tomates Usando Técnicas de Visión Artificial



VISION ARTIFICIAL

INGENIRIA EN COMPUTACION

Autores:

- RAMIRO VEGA MEZA

Profesor:

Vianney Muñoz Jiménez

INDICE

1. introducción	4
2. Descripción del Sistema	5
3. Arquitectura y Flujo de Funcionamiento	6
Imagen 3.1 arquitectura del sistema	6
4. Dataset Robot Flow	8
Imagen 4.1 dataset robotflow	8
Imagen 4.2 clase defectuoso	9
Imagen 4.3 clase maduro	10
Imagen 4.4 clase verde	10
Imagen 4.4 conteo de clases	11
5. Modelo de Detección (YOLOv8)	12
5.1 Flujo General de YOLOv8	13
Imagen 5.1 como funciona yolo	13
5.2 Entrenamiento del modelo	14
Imagen 5.2 codigo train.py	14
5.3 Resultados del entrenamiento	15
Imagen 5.3.1.1 Precision Recall Curve	15
Imagen 5.3.1.2 F1 Confidence Curve	16
5.3.2 Métricas generales	16
Imagen 5.3.2.1 Resultados metricas	17
Imagen 5.3.2.2 csv de resultados	17
5.3.3 Matriz de confusión	18
Imagen 5.3.3.1 matriz de confusion	18
Imagen 5.3.3.2 matriz de confusion normalizada	19
5.3.4 Resultados visuales	19
Imagen 5.3.4.1 train batch 0	20
Imagen 5.3.4.2 train batch 1	21
Imagen 5.3.4.2 train batch 2	22
Imagen 5.3.4.2 train batch 1440	23
Imagen 5.3.4.2 train batch 1440	23
6. Tracking de Objetos (SORT)	24
7. Evaluación del Modelo	24
Imagen 7.1 codigo para evaluar	25
Tabla 7.2 Metricas de evaluacion	25
8. Implementación del Sistema	25

Imagen 8.1 implementacion	26
9. Resultados del Sistema	27
Imagen 10.1 aplicación corriendo	28
11. Conclusiones	29
12. Referencias	30

1. introducción

En los últimos años, la incorporación de tecnologías basadas en inteligencia artificial ha transformado diversos sectores productivos, especialmente el ámbito agrícola, donde la optimización de procesos y la mejora en la calidad de los productos son fundamentales. La automatización de tareas tradicionales, como la inspección visual de cultivos, permite reducir errores humanos y aumentar la eficiencia en los procesos de selección y clasificación. En este contexto, la visión por computadora surge como una herramienta clave para analizar imágenes en tiempo real y generar información útil para la toma de decisiones.

El presente proyecto, denominado AgroVision, tiene como objetivo el desarrollo de un sistema inteligente capaz de detectar y clasificar tomates en tiempo real utilizando técnicas de aprendizaje profundo. Para ello, se emplea el modelo YOLOv8 (You Only Look Once), el cual permite identificar objetos dentro de una imagen de forma rápida y precisa. El sistema ha sido entrenado para reconocer tres tipos de tomates: maduros, verdes y defectuosos, lo que facilita la automatización de procesos de inspección en entornos agrícolas.

Con el fin de mejorar la consistencia de las detecciones a lo largo del tiempo, se incorpora un sistema de seguimiento basado en el algoritmo SORT (Simple Online and Realtime Tracking). Este componente permite asignar un identificador único a cada tomate detectado, evitando duplicaciones y permitiendo llevar un conteo más preciso. De esta manera, el sistema no solo detecta objetos, sino que también mantiene un seguimiento continuo de los mismos durante la transmisión de video.

Adicionalmente, AgroVision integra un dashboard web interactivo desarrollado con Flask, el cual permite visualizar en tiempo real la información generada por el sistema. Este dashboard muestra datos como el número total de tomates detectados y su clasificación por categoría, proporcionando una interfaz clara y accesible para el usuario. La combinación de procesamiento en tiempo real y visualización dinámica facilita la supervisión del sistema desde cualquier navegador.

Finalmente, este proyecto demuestra la viabilidad de integrar técnicas de visión por computadora, aprendizaje automático y desarrollo web en una única solución funcional. AgroVision representa un ejemplo práctico de cómo las tecnologías modernas pueden aplicarse para mejorar procesos agrícolas, ofreciendo una herramienta eficiente, escalable y adaptable a diferentes escenarios. Asimismo, sienta las bases para futuras mejoras y aplicaciones en otros tipos de cultivos o entornos industriales.

2. Descripción del Sistema

El sistema AgroVision es una solución integral de visión por computadora diseñada para la detección, clasificación y seguimiento de tomates en tiempo real mediante el uso de técnicas de aprendizaje profundo. Su objetivo principal es automatizar el proceso de inspección visual, reduciendo la intervención humana y mejorando la eficiencia en entornos agrícolas o de clasificación de productos.

El funcionamiento del sistema se basa en la captura continua de imágenes a través de una cámara, las cuales son procesadas en tiempo real. Cada cuadro capturado es enviado a un modelo de detección basado en YOLOv8, el cual identifica los objetos presentes en la imagen y los clasifica según su estado: maduro, verde o defectuoso. Este proceso permite detectar múltiples tomates simultáneamente con alta precisión y velocidad.

Una vez realizadas las detecciones, el sistema incorpora un módulo de seguimiento de objetos utilizando el algoritmo SORT, el cual permite asignar un identificador único a cada tomate detectado. Este seguimiento continuo garantiza que cada objeto pueda ser identificado a lo largo del tiempo, evitando duplicidades en el conteo y proporcionando mayor coherencia en los resultados obtenidos durante la ejecución.

Como complemento a la detección y el seguimiento, AgroVision cuenta con un módulo de visualización que muestra los resultados directamente sobre la imagen procesada. En esta visualización se incluyen recuadros delimitadores (bounding boxes), etiquetas con el identificador del objeto, su estado y el nivel de confianza de la detección. Además, el sistema calcula y muestra métricas como los fotogramas por segundo (FPS), lo que permite evaluar el rendimiento en tiempo real.

Finalmente, el sistema integra un dashboard web interactivo, desarrollado con Flask, el cual recibe la información procesada y permite su visualización de manera estructurada. Este dashboard muestra el número total de tomates detectados, así como su clasificación por categoría, facilitando la supervisión del sistema de forma remota. De este modo, AgroVision se presenta como una solución completa que combina captura de datos, procesamiento inteligente y visualización en tiempo real en una única plataforma.

3. Arquitectura y Flujo de Funcionamiento

La arquitectura del sistema AgroVision está diseñada de manera modular, lo que permite integrar diferentes componentes especializados en el procesamiento de información visual en tiempo real. Cada módulo cumple una función específica dentro del sistema, facilitando tanto el desarrollo como el mantenimiento y la escalabilidad del proyecto. Esta organización modular se compone principalmente de los módulos de captura de video, detección, seguimiento, procesamiento de datos y visualización.



Imagen 3.1 arquitectura del sistema

En primer lugar, el sistema inicia con el módulo de captura de imágenes, el cual utiliza una cámara para obtener frames en tiempo real. Cada uno de estos frames es enviado directamente al siguiente componente sin almacenamiento intermedio, lo que permite reducir la latencia y mantener un procesamiento continuo. Este flujo constante de datos es fundamental para lograr una respuesta rápida y eficiente del sistema durante su ejecución.

Posteriormente, los frames capturados son procesados por el módulo de detección basado en YOLOv8, donde se identifican los objetos presentes en la imagen junto con su clasificación. Este módulo devuelve información estructurada que incluye las coordenadas de las cajas delimitadoras, la clase detectada y el nivel de confianza. La rapidez y precisión de este modelo permiten analizar múltiples objetos en un mismo frame sin comprometer el rendimiento.

Una vez obtenidas las detecciones, el sistema emplea el módulo de seguimiento de objetos mediante el algoritmo SORT, el cual asocia las detecciones entre diferentes frames y asigna un identificador único a cada objeto. Este proceso asegura que cada tomate pueda ser rastreado a lo largo del tiempo, permitiendo mantener una persistencia en los resultados y evitando duplicaciones en el conteo.

Finalmente, todos los datos procesados son utilizados en el módulo de visualización y dashboard web, donde se presentan tanto en la ventana de ejecución (con bounding boxes

y etiquetas) como en un panel web interactivo. El flujo completo del sistema puede resumirse de la siguiente manera: captura de imagen → detección → tracking → clasificación → visualización → actualización del dashboard. Esta arquitectura permite que AgroVision opere de manera fluida, eficiente y en tiempo real, cumpliendo con los objetivos planteados en el proyecto.

4. Dataset Robot Flow

El conjunto de datos utilizado en el proyecto AgroVision fue desarrollado y gestionado mediante la plataforma Roboflow, la cual permite la creación, anotación y organización de datasets para tareas de visión por computadora. Este dataset está compuesto por imágenes de tomates capturadas en diferentes condiciones, incluyendo variaciones de iluminación, posición y número de objetos en escena.

Las imágenes fueron etiquetadas manualmente mediante el uso de cajas delimitadoras (bounding boxes), con el objetivo de identificar la ubicación exacta de cada tomate dentro de la imagen. Este proceso de anotación es fundamental para el entrenamiento del modelo de detección, ya que permite al algoritmo aprender a reconocer patrones visuales asociados a cada clase.

El dataset se encuentra organizado en las divisiones estándar de entrenamiento, validación y prueba, siguiendo la estructura del formato YOLO. Esta organización facilita el entrenamiento y evaluación del modelo, asegurando que el sistema pueda generalizar correctamente ante nuevos datos no vistos previamente.

En cuanto a la clasificación, el dataset incluye tres clases principales: maduro, verde y defectuoso. Cada clase representa un estado distinto del tomate, lo cual permite realizar una clasificación funcional para aplicaciones reales. El número de instancias por clase es relativamente equilibrado, con aproximadamente 303 muestras para la clase maduro, 220 para verde y 202 para defectuoso, lo que contribuye a evitar sesgos durante el entrenamiento.

Finalmente, el dataset fue exportado en formato compatible con YOLOv8, incluyendo las etiquetas en archivos de texto que contienen las coordenadas normalizadas de cada objeto. Esta preparación permite una integración directa con el modelo de detección utilizado en el proyecto, garantizando un flujo de trabajo eficiente desde la recopilación de datos hasta la implementación del sistema.

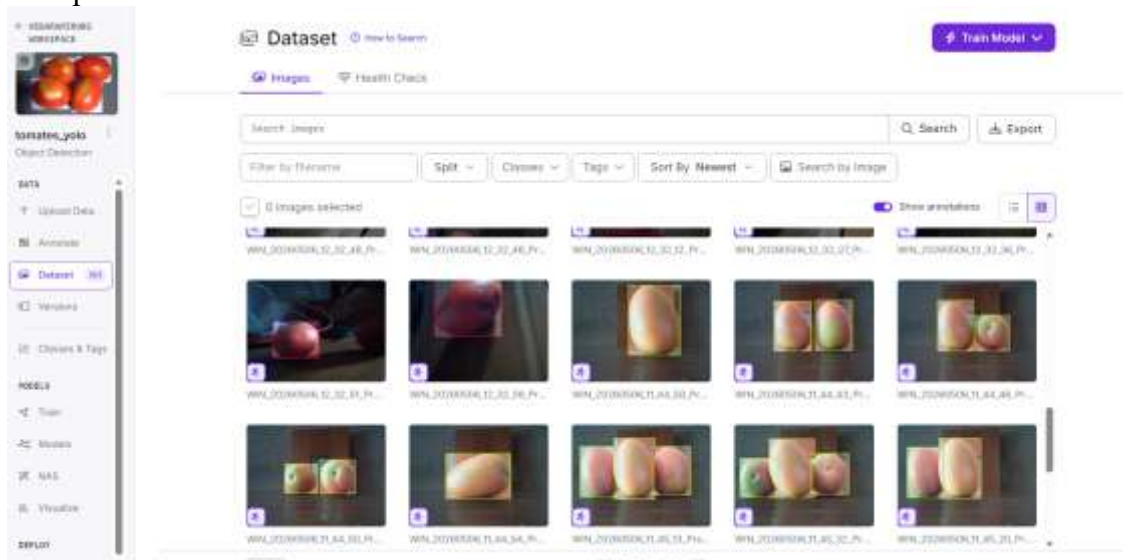


Imagen 4.1 dataset robotflow

Para la clase defectuoso, se seleccionaron imágenes de tomates que presentan imperfecciones visibles, tales como manchas oscuras, zonas con decoloración o indicios de deterioro. Estas características incluyen principalmente la presencia de áreas negras o signos iniciales de descomposición, los cuales representan defectos comunes en productos agrícolas durante su proceso de maduración o almacenamiento.

La inclusión de este tipo de ejemplos dentro del dataset permite al modelo aprender a identificar condiciones no óptimas del fruto, lo cual resulta fundamental para aplicaciones prácticas en procesos de inspección y clasificación. De esta manera, el sistema no solo reconoce tomates en buen estado, sino que también es capaz de detectar aquellos que podrían considerarse no aptos para consumo o comercialización.

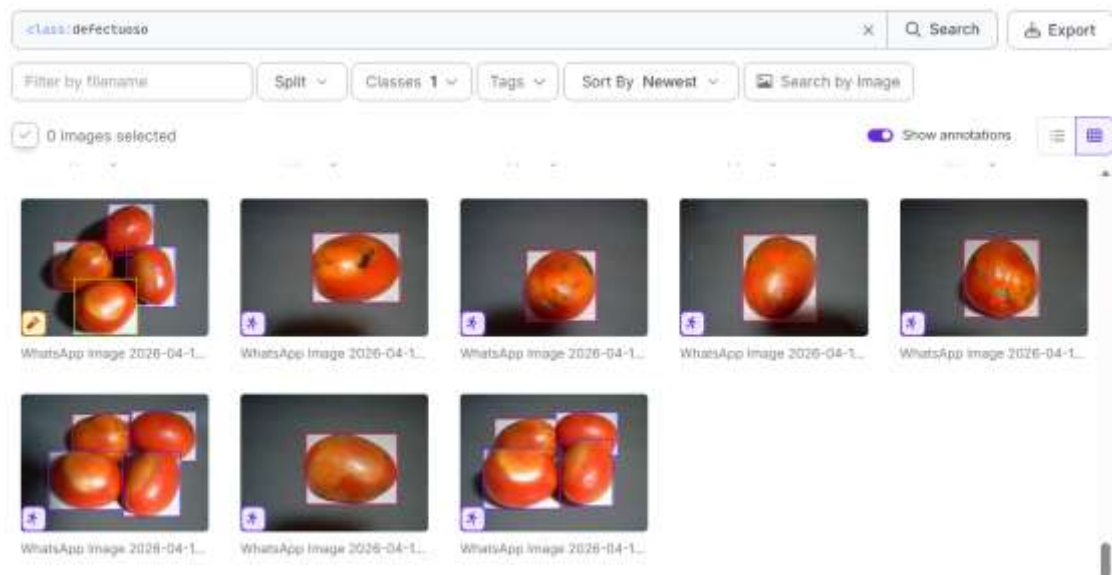


Imagen 4.2 clase defectuoso

Para la clase maduro, se seleccionaron imágenes de tomates que presentan un grado avanzado de maduración, caracterizados principalmente por un color rojo intenso y uniforme en la superficie del fruto. Se priorizó la inclusión de ejemplos donde el tomate mostrara la mayor cantidad posible de tonalidades rojas, evitando aquellos con zonas verdes visibles o mezclas de color que pudieran generar ambigüedad en la clasificación.

Este criterio de selección permite que el modelo aprenda de manera clara las características visuales asociadas a un tomate en estado óptimo de maduración. La homogeneidad en el color facilita la diferenciación respecto a otras clases, especialmente frente a la clase "verde", mejorando así la precisión del sistema en la tarea de clasificación.

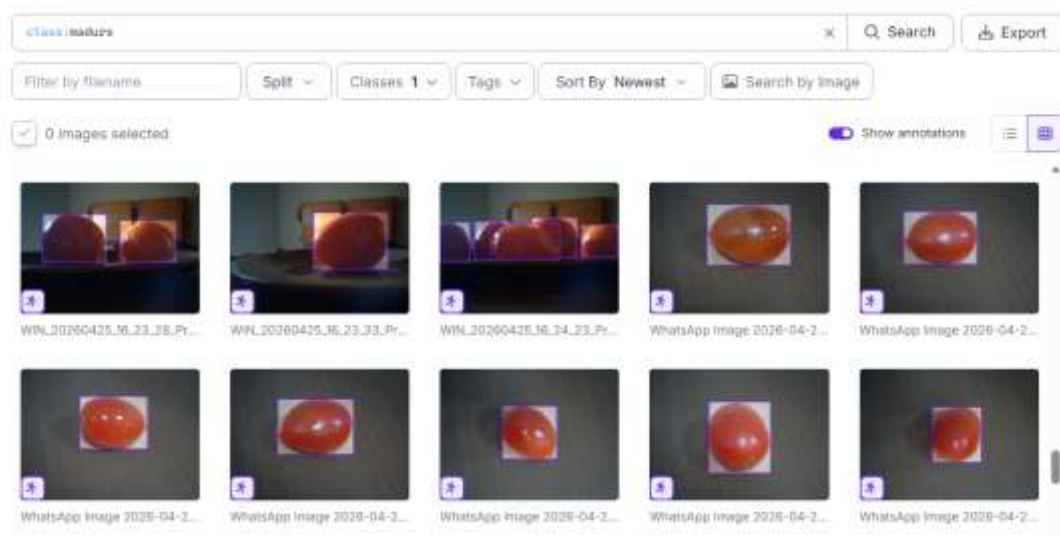


Imagen 4.3 clase maduro

Para la clase verde, se seleccionaron imágenes de tomates que presentan un estado de maduración incompleto, caracterizados principalmente por un color verde predominante en la superficie del fruto. Asimismo, se incluyeron ejemplos de tomates que, aunque puedan presentar algunas zonas rojizas, conservan manchas o áreas verdes visibles, lo que indica que aún no han alcanzado su madurez total.

La inclusión de estas variaciones dentro de la clase permite al modelo aprender a identificar diferentes grados de inmadurez, evitando confusiones con la clase "maduro". Este criterio resulta importante, ya que en situaciones reales los tomates pueden encontrarse en etapas intermedias de maduración, por lo que el sistema debe ser capaz de reconocer dichas diferencias de manera precisa.

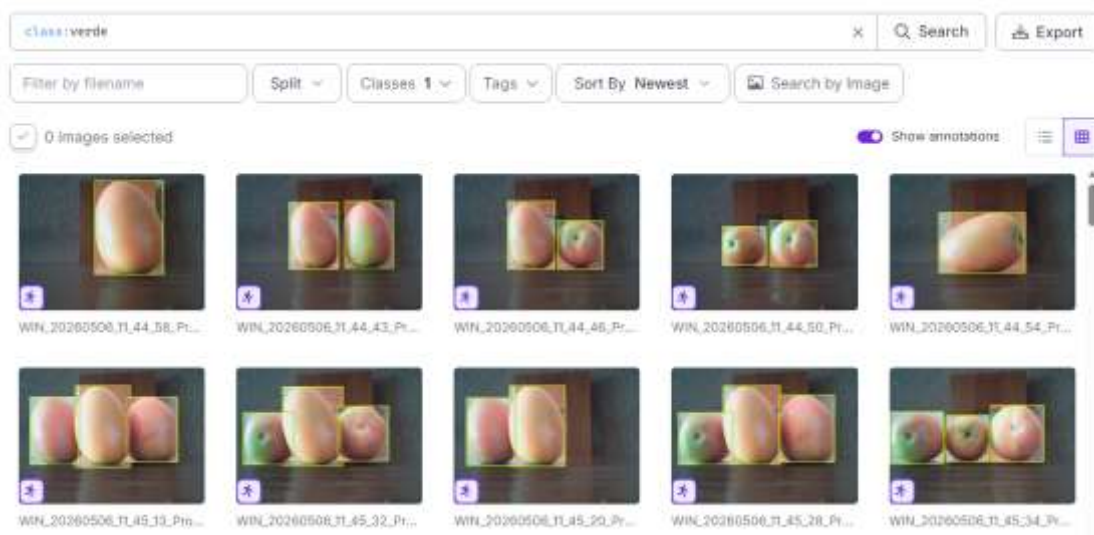


Imagen 4.4 clase verde

El conteo de instancias dentro del dataset se realiza a partir del número total de objetos etiquetados, y no únicamente del número de imágenes. Esto se debe a que una sola imagen

puede contener uno o varios tomates, cada uno con su respectiva etiqueta y clase asociada. En este sentido, el sistema de anotación contabiliza cada objeto de manera independiente, lo que permite obtener una representación más precisa de la distribución de datos.

Como se observa, el dataset presenta una distribución relativamente equilibrada entre las tres categorías. Este equilibrio es importante para el entrenamiento del modelo, ya que evita sesgos hacia alguna clase en particular.

Es importante destacar que muchas de las imágenes utilizadas contienen múltiples objetos, por lo que en una sola imagen pueden aparecer tomates pertenecientes a una o incluso varias clases distintas. Por ejemplo, es común encontrar escenas donde se observan tomates maduros junto con tomates verdes o defectuosos, lo cual incrementa la complejidad del problema y mejora la capacidad del modelo para generalizar en situaciones reales.

Esta característica del dataset permite que el sistema aprenda a realizar detecciones simultáneas y clasificaciones múltiples dentro de una misma imagen, acercándose a condiciones reales de operación donde los objetos no se presentan de manera aislada. Como resultado, el modelo desarrollado a partir de este conjunto de datos es más robusto y capaz de manejar escenarios con alta variabilidad.

Search classes...

Sort By Class Ascending

COLOR	CLASS NAME	COUNT
	defectuoso	282
	maduro	383
	verde	228

Imagen 4.4 conteo de clases

5. Modelo de Detección (YOLOv8)

El sistema AgroVision utiliza como núcleo de detección el modelo YOLOv8 (You Only Look Once versión 8), un algoritmo de aprendizaje profundo ampliamente reconocido por su alta precisión y velocidad en tareas de detección de objetos en tiempo real. Este modelo se basa en redes neuronales convolucionales (CNN), las cuales permiten analizar imágenes completas en una sola pasada, identificando simultáneamente múltiples objetos y sus respectivas clases.

YOLOv8 pertenece a una familia de modelos optimizados para aplicaciones en tiempo real, ya que realiza la detección de manera directa sobre la imagen sin necesidad de dividirla en múltiples regiones. Esta característica lo hace especialmente eficiente, permitiendo procesar imágenes con baja latencia y alto rendimiento, lo cual resulta ideal para el sistema AgroVision, donde se requiere analizar frames de video de forma continua.

Para el desarrollo de este proyecto, se utilizó un modelo preentrenado (yolov8n.pt), el cual fue posteriormente ajustado mediante un proceso de entrenamiento personalizado utilizando el dataset de tomates. Este enfoque, conocido como transfer learning, permite aprovechar el conocimiento previamente adquirido por el modelo en tareas generales, reduciendo el tiempo de entrenamiento y mejorando la capacidad de detección en el nuevo contexto.

Durante el proceso de entrenamiento, el modelo fue configurado con parámetros específicos como el número de épocas, tamaño de imagen de entrada y tamaño de lote, lo cual permitió optimizar el aprendizaje en función de los recursos disponibles. El modelo fue entrenado para identificar las tres clases definidas en el dataset: maduro, verde y defectuoso, aprendiendo a reconocer sus características visuales a partir de las anotaciones proporcionadas.

El resultado de este proceso es un modelo capaz de detectar tomates en imágenes en tiempo real, generando como salida un conjunto de predicciones que incluyen las coordenadas de las cajas delimitadoras, la clase asignada y el nivel de confianza de cada detección. Estas predicciones son posteriormente utilizadas por el sistema para realizar el seguimiento de objetos, la visualización en pantalla y la actualización del dashboard, integrándose de manera eficiente en el flujo general del sistema AgroVision.

5.1 Flujo General de YOLOv8

La imagen muestra cómo funciona YOLOv8 para reconocer objetos en una imagen o video. Primero, el sistema recibe una imagen y comienza a analizarla poco a poco para identificar formas, colores y patrones importantes, como si “aprendiera a ver” los objetos. Después, combina toda esa información para encontrar objetos de distintos tamaños y ubicaciones dentro de la imagen. Finalmente, el modelo decide qué objeto encontró, dónde está y qué tan seguro está de la detección, dibujando un recuadro alrededor del objeto identificado. En proyectos como AgroVision, este proceso permite detectar tomates y clasificarlos automáticamente como maduros, verdes o defectuosos en tiempo real.

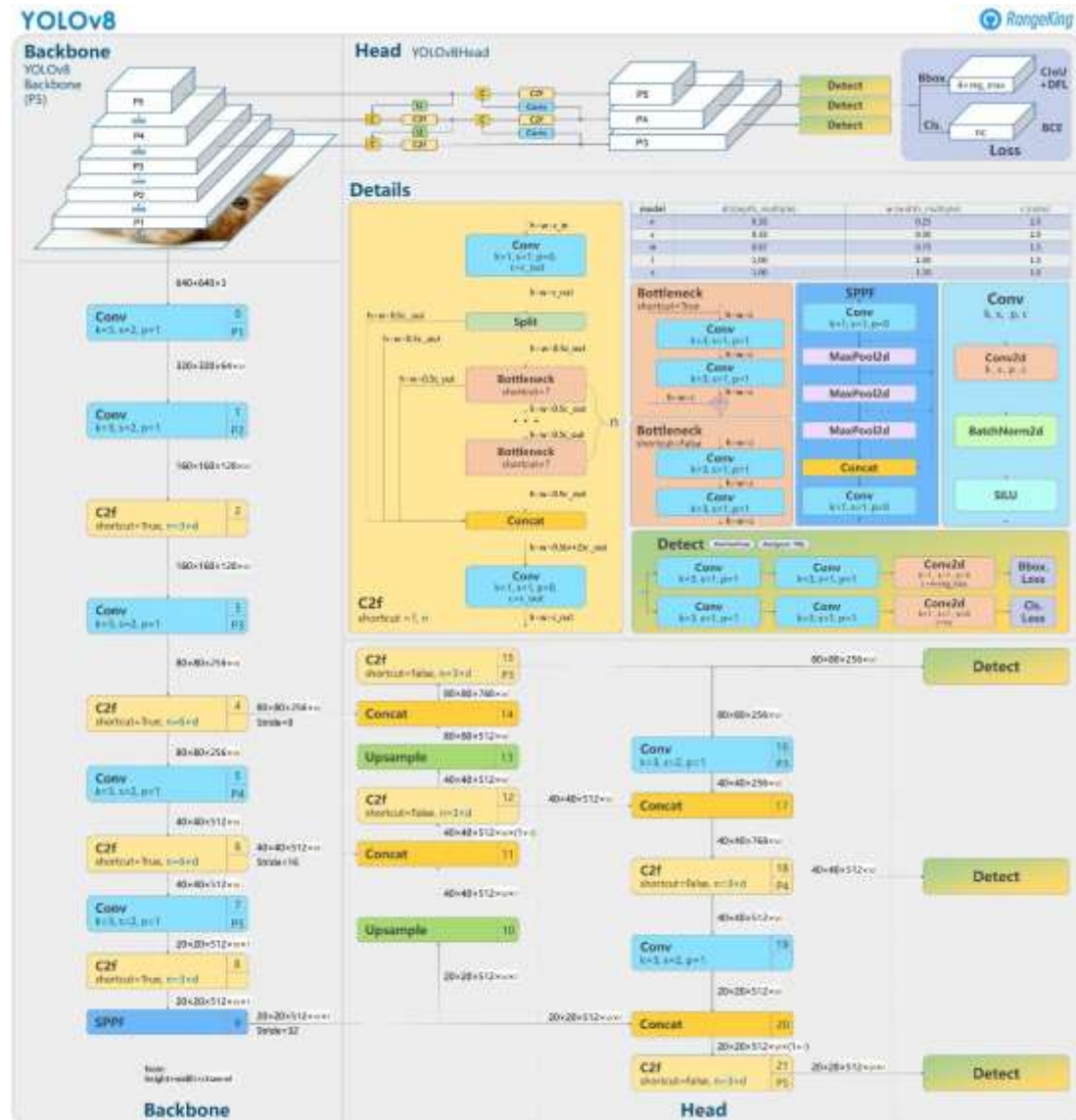
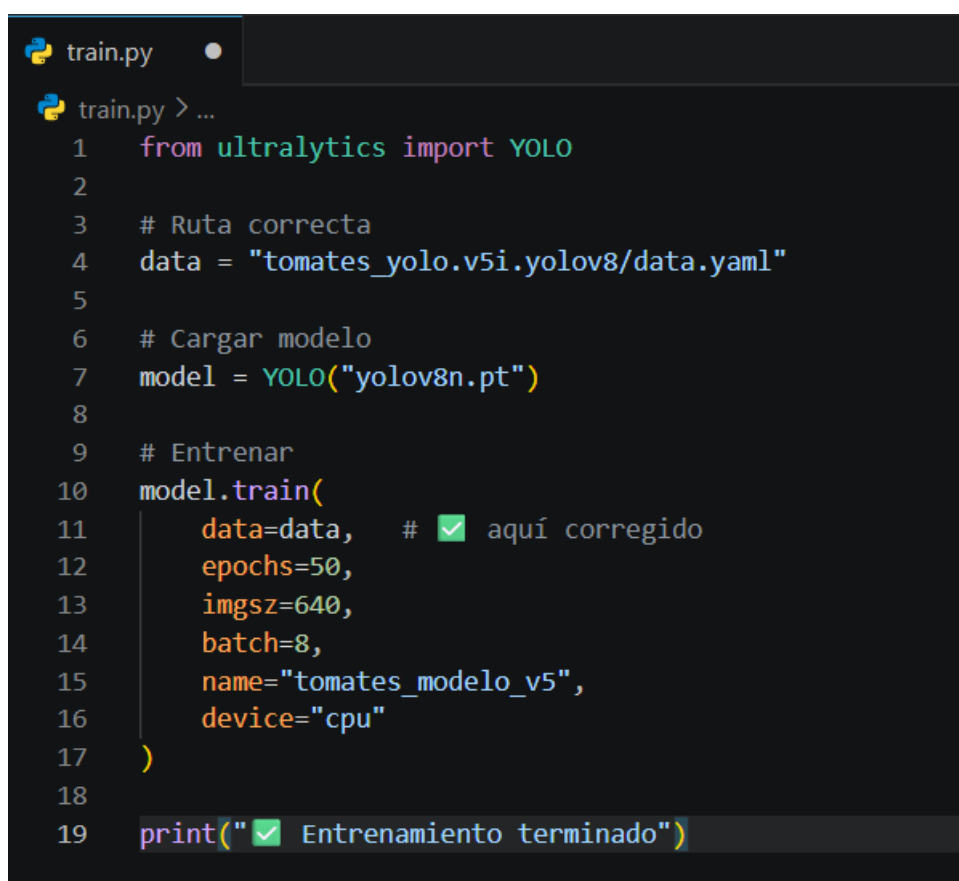


Imagen 5.1 como funciona yolo

5.2 Entrenamiento del modelo

El código presentado corresponde al proceso de entrenamiento del modelo de detección YOLOv8 utilizando la librería Ultralytics. En primer lugar, se importa la clase YOLO y se define la ruta al archivo data.yaml, el cual contiene la configuración del dataset, incluyendo las rutas de las imágenes y las clases a detectar.

Posteriormente, se carga un modelo preentrenado (yolov8n.pt), que sirve como base para aplicar la técnica de transfer learning. A continuación, se ejecuta el método train(), donde se especifican los parámetros de entrenamiento, tales como el número de épocas (50), el tamaño de las imágenes de entrada (640 píxeles), el tamaño del lote (8) y el dispositivo de ejecución (CPU). Además, se asigna un nombre al experimento para organizar los resultados generados. Finalmente, una vez completado el proceso, se muestra un mensaje indicando que el entrenamiento ha finalizado correctamente, lo que implica que el modelo ha aprendido a identificar las clases definidas en el dataset.



```
train.py
train.py > ...
1  from ultralytics import YOLO
2
3  # Ruta correcta
4  data = "tomates_yolo.v5i.yolov8/data.yaml"
5
6  # Cargar modelo
7  model = YOLO("yolov8n.pt")
8
9  # Entrenar
10 model.train(
11     data=data, # ✅ aquí corregido
12     epochs=50,
13     imgsz=640,
14     batch=8,
15     name="tomates_modelo_v5",
16     device="cpu"
17 )
18
19 print("✅ Entrenamiento terminado")
```

Imagen 5.2 código train.py

5.3 Resultados del entrenamiento

5.3.1 Curvas de desempeño

La gráfica mostrada corresponde a la curva Precision-Recall, una de las métricas más importantes en modelos de detección de objetos. En esta curva, el eje horizontal representa el recall (sensibilidad), que mide la capacidad del modelo para detectar todos los objetos reales, mientras que el eje vertical representa la precisión (precision), que indica qué tan correctas son las detecciones realizadas. Idealmente, se busca que ambos valores sean cercanos a 1, lo que indica un desempeño óptimo. En este caso, la gráfica parece “vacía” debido a que las curvas están prácticamente en el valor máximo (cerca a 0.995), lo que significa que el modelo alcanzó un rendimiento muy alto. Esto provoca que las líneas se superpongan en la parte superior de la gráfica, dando la impresión de que no hay curva visible. Asimismo, el valor de $mAP@0.5 = 0.995$ indica que el modelo tiene una excelente capacidad tanto para localizar como para clasificar correctamente los objetos, lo que confirma un desempeño sobresaliente en el dataset utilizado.

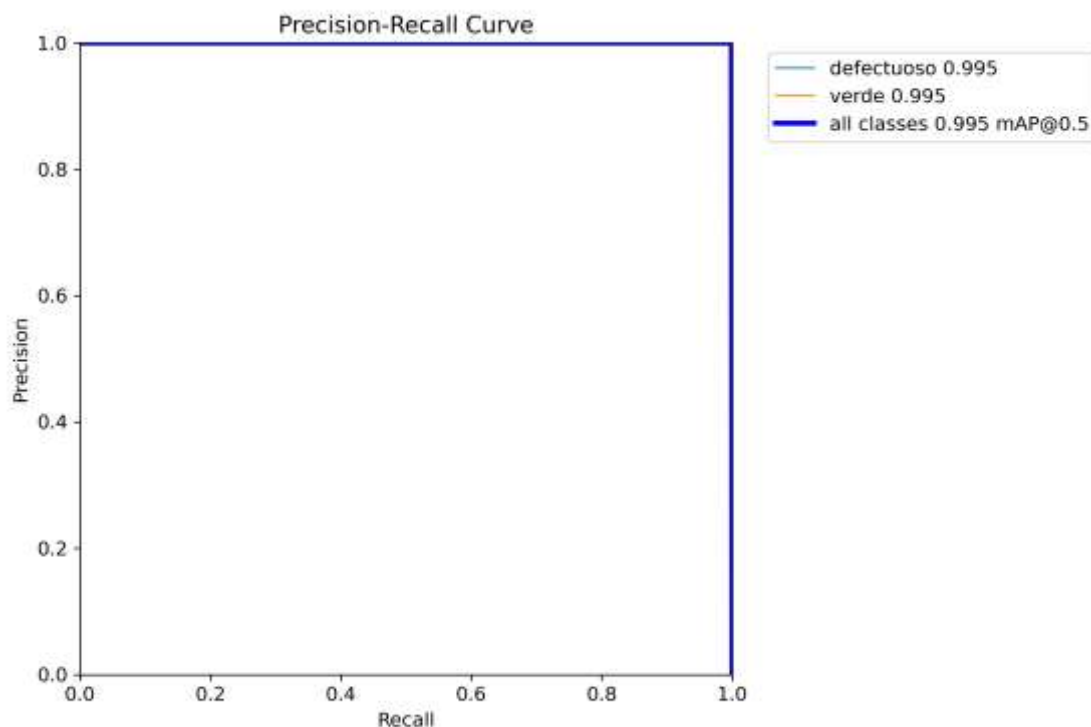


Imagen 5.3.1.1 Precision Recall Curve

La gráfica muestra cómo cambia el F1-score del modelo en función del umbral de confianza de las predicciones. Se observa que, para valores bajos de confianza (cerca de 0), el F1 es bajo porque el modelo acepta casi todas las predicciones y comete muchos errores; conforme aumenta la confianza, el F1 mejora hasta alcanzar un máximo cercano a 0.98 alrededor de un umbral de 0.72, lo que indica el mejor equilibrio entre precisión y recall. Después de ese punto, al exigir una confianza demasiado alta (cerca de 1.0), el F1 cae bruscamente porque el modelo se vuelve demasiado estricto y deja de detectar muchos casos positivos. Las curvas de las clases “defectuoso” y “verde” siguen un patrón similar,

aunque con ligeras diferencias, lo que indica que el modelo funciona de forma consistente para ambas clases, siendo ese valor de ~ 0.72 el umbral óptimo para las predicciones.

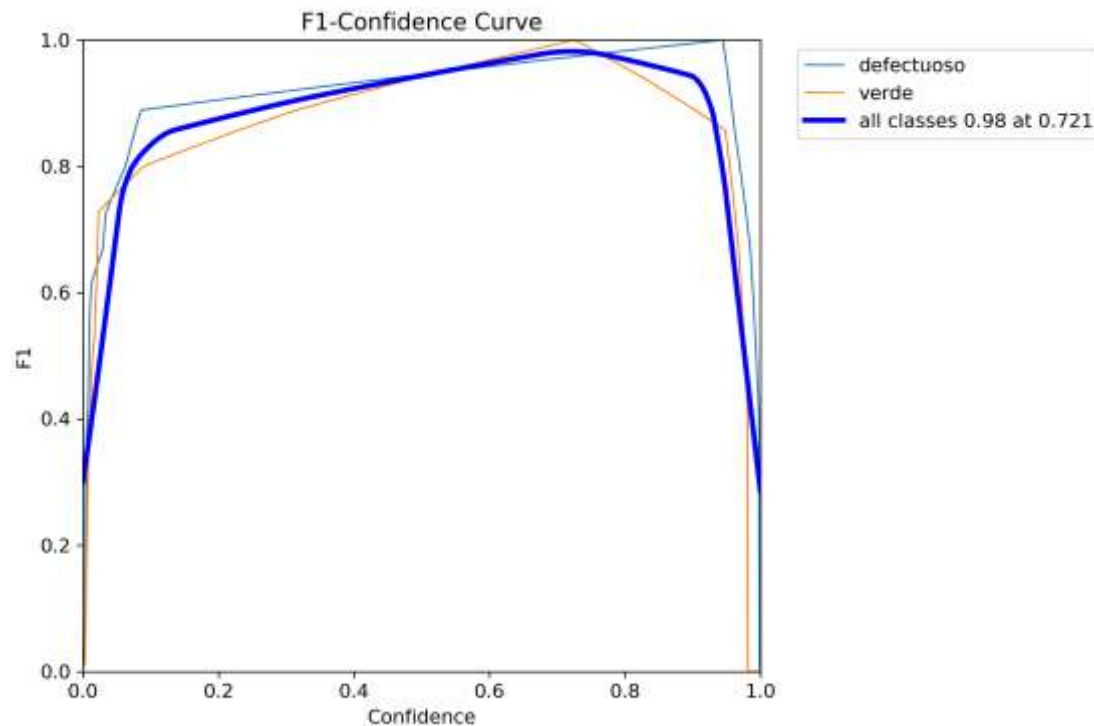


Imagen 5.3.1.2 F1 Confidence Curve

5.3.2 Métricas generales

La gráfica de resultados muestra la evolución del modelo a lo largo del entrenamiento, incluyendo métricas clave como la pérdida (loss), la precisión (precision) y el recall. Estas curvas permiten observar cómo el modelo mejora su desempeño conforme avanza el número de épocas. Generalmente, una disminución progresiva de la pérdida indica que el modelo está aprendiendo correctamente, mientras que un incremento en precisión y recall refleja una mejor capacidad para identificar objetos. En conjunto, esta gráfica permite evaluar la estabilidad del entrenamiento y verificar que el modelo no presenta problemas como sobreajuste o subentrenamiento.

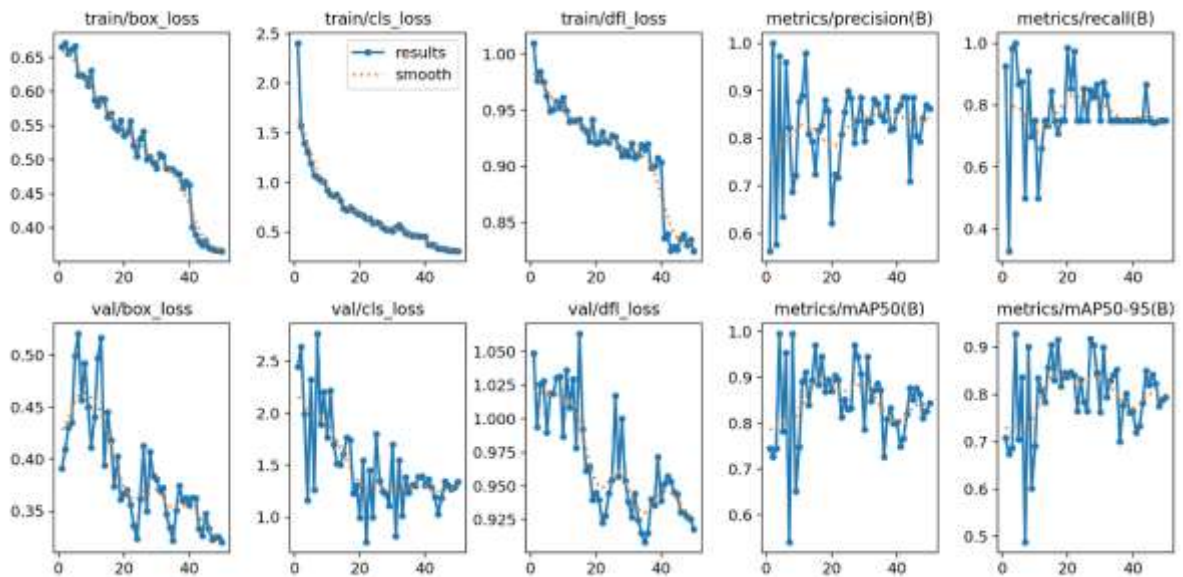


Imagen 5.3.2.1 Resultados metricas

El archivo results.csv contiene un registro numérico detallado del proceso de entrenamiento del modelo, donde se almacenan las métricas obtenidas en cada época. Entre los valores registrados se incluyen la pérdida del modelo (loss), la precisión, el recall y el mAP, lo que permite analizar de manera cuantitativa el comportamiento del entrenamiento. Este archivo resulta especialmente útil para realizar análisis más profundos o generar gráficas personalizadas, ya que permite observar con exactitud cómo evoluciona el rendimiento del modelo a lo largo del tiempo. En conjunto, el results.csv complementa las gráficas visuales, proporcionando evidencia numérica del aprendizaje del modelo y facilitando la validación de sus resultados.

```

epoch,train_box_loss,train_cls_loss,train_dfl_loss,metrics_precision(B),metrics_recall(B),metrics_mAP50(B),metrics_mAP50_95(B)
0 1.0000, 1.0000, 1.0000, 0.0000, 0.0000, 0.0000, 0.0000
1 0.9999, 0.9999, 0.9999, 0.0000, 0.0000, 0.0000, 0.0000
2 0.9998, 0.9998, 0.9998, 0.0000, 0.0000, 0.0000, 0.0000
3 0.9997, 0.9997, 0.9997, 0.0000, 0.0000, 0.0000, 0.0000
4 0.9996, 0.9996, 0.9996, 0.0000, 0.0000, 0.0000, 0.0000
5 0.9995, 0.9995, 0.9995, 0.0000, 0.0000, 0.0000, 0.0000
6 0.9994, 0.9994, 0.9994, 0.0000, 0.0000, 0.0000, 0.0000
7 0.9993, 0.9993, 0.9993, 0.0000, 0.0000, 0.0000, 0.0000
8 0.9992, 0.9992, 0.9992, 0.0000, 0.0000, 0.0000, 0.0000
9 0.9991, 0.9991, 0.9991, 0.0000, 0.0000, 0.0000, 0.0000
10 0.9990, 0.9990, 0.9990, 0.0000, 0.0000, 0.0000, 0.0000
11 0.9989, 0.9989, 0.9989, 0.0000, 0.0000, 0.0000, 0.0000
12 0.9988, 0.9988, 0.9988, 0.0000, 0.0000, 0.0000, 0.0000
13 0.9987, 0.9987, 0.9987, 0.0000, 0.0000, 0.0000, 0.0000
14 0.9986, 0.9986, 0.9986, 0.0000, 0.0000, 0.0000, 0.0000
15 0.9985, 0.9985, 0.9985, 0.0000, 0.0000, 0.0000, 0.0000
16 0.9984, 0.9984, 0.9984, 0.0000, 0.0000, 0.0000, 0.0000
17 0.9983, 0.9983, 0.9983, 0.0000, 0.0000, 0.0000, 0.0000
18 0.9982, 0.9982, 0.9982, 0.0000, 0.0000, 0.0000, 0.0000
19 0.9981, 0.9981, 0.9981, 0.0000, 0.0000, 0.0000, 0.0000
20 0.9980, 0.9980, 0.9980, 0.0000, 0.0000, 0.0000, 0.0000
21 0.9979, 0.9979, 0.9979, 0.0000, 0.0000, 0.0000, 0.0000
22 0.9978, 0.9978, 0.9978, 0.0000, 0.0000, 0.0000, 0.0000
23 0.9977, 0.9977, 0.9977, 0.0000, 0.0000, 0.0000, 0.0000
24 0.9976, 0.9976, 0.9976, 0.0000, 0.0000, 0.0000, 0.0000
25 0.9975, 0.9975, 0.9975, 0.0000, 0.0000, 0.0000, 0.0000
26 0.9974, 0.9974, 0.9974, 0.0000, 0.0000, 0.0000, 0.0000
27 0.9973, 0.9973, 0.9973, 0.0000, 0.0000, 0.0000, 0.0000
28 0.9972, 0.9972, 0.9972, 0.0000, 0.0000, 0.0000, 0.0000
29 0.9971, 0.9971, 0.9971, 0.0000, 0.0000, 0.0000, 0.0000
30 0.9970, 0.9970, 0.9970, 0.0000, 0.0000, 0.0000, 0.0000
31 0.9969, 0.9969, 0.9969, 0.0000, 0.0000, 0.0000, 0.0000
32 0.9968, 0.9968, 0.9968, 0.0000, 0.0000, 0.0000, 0.0000
33 0.9967, 0.9967, 0.9967, 0.0000, 0.0000, 0.0000, 0.0000
34 0.9966, 0.9966, 0.9966, 0.0000, 0.0000, 0.0000, 0.0000
35 0.9965, 0.9965, 0.9965, 0.0000, 0.0000, 0.0000, 0.0000
36 0.9964, 0.9964, 0.9964, 0.0000, 0.0000, 0.0000, 0.0000
37 0.9963, 0.9963, 0.9963, 0.0000, 0.0000, 0.0000, 0.0000
38 0.9962, 0.9962, 0.9962, 0.0000, 0.0000, 0.0000, 0.0000
39 0.9961, 0.9961, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000
40 0.9960, 0.9960, 0.9960, 0.0000, 0.0000, 0.0000, 0.0000

```

Imagen 5.3.2.2 csv de resultados

5.3.3 Matriz de confusión

En la matriz de confusión podemos observar cómo nuestro modelo clasifica cada una de las clases al comparar las etiquetas reales con las predicciones realizadas. Se aprecia que para la clase “defectuoso” el modelo logra 4 aciertos correctos, lo que indica un buen desempeño en esta categoría. Para la clase “verde”, también obtenemos 4 aciertos, aunque se presenta 1 error, donde un elemento de otra clase fue clasificado incorrectamente como “verde”. Por otro lado, no se registran resultados en las clases “maduro” y “background”, lo que sugiere que el modelo no está reconociendo estas clases o que el conjunto de datos podría estar desbalanceado. En conjunto, interpretamos que el modelo tiene un buen rendimiento en las clases principales, pero aún requiere mejoras para abarcar todas las categorías de manera más equilibrada.

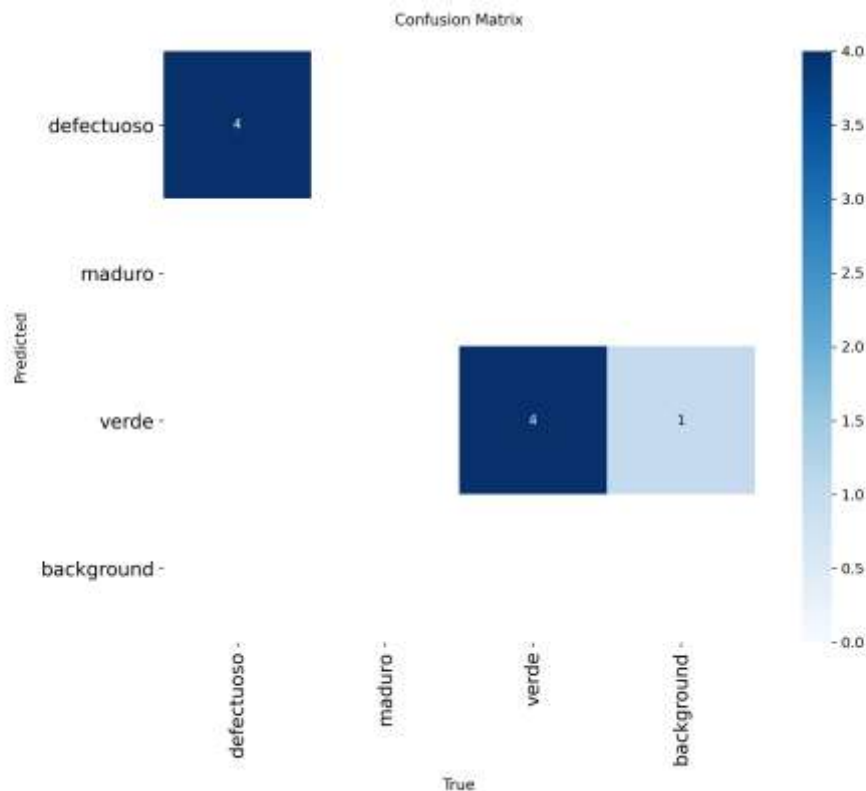


Imagen 5.3.3.1 matriz de confusion

En la matriz de confusión normalizada podemos observar el desempeño de nuestro modelo en términos proporcionales, es decir, cada valor representa el porcentaje de aciertos o errores respecto al total de ejemplos de cada clase. Se aprecia que la clase “defectuoso” tiene un valor de 1.00, lo que indica que el 100% de los casos reales de esta categoría fueron correctamente clasificados. De manera similar, la clase “verde” también presenta un valor de 1.00 en su categoría correcta, lo que refleja un rendimiento perfecto en términos relativos; sin embargo, también aparece un valor de 1.00 en otra posición, lo que sugiere que el único error observado en la matriz original representa el 100% de los casos de esa categoría específica, probablemente debido a un bajo número de muestras. Por otro lado, las clases “maduro” y “background” no muestran valores, lo que indica ausencia de datos o que el modelo no realizó predicciones para ellas. En conjunto, interpretamos que el modelo presenta un desempeño muy alto en las clases evaluadas, aunque la normalización evidencia

posibles limitaciones por el tamaño reducido o desbalance del conjunto de datos.

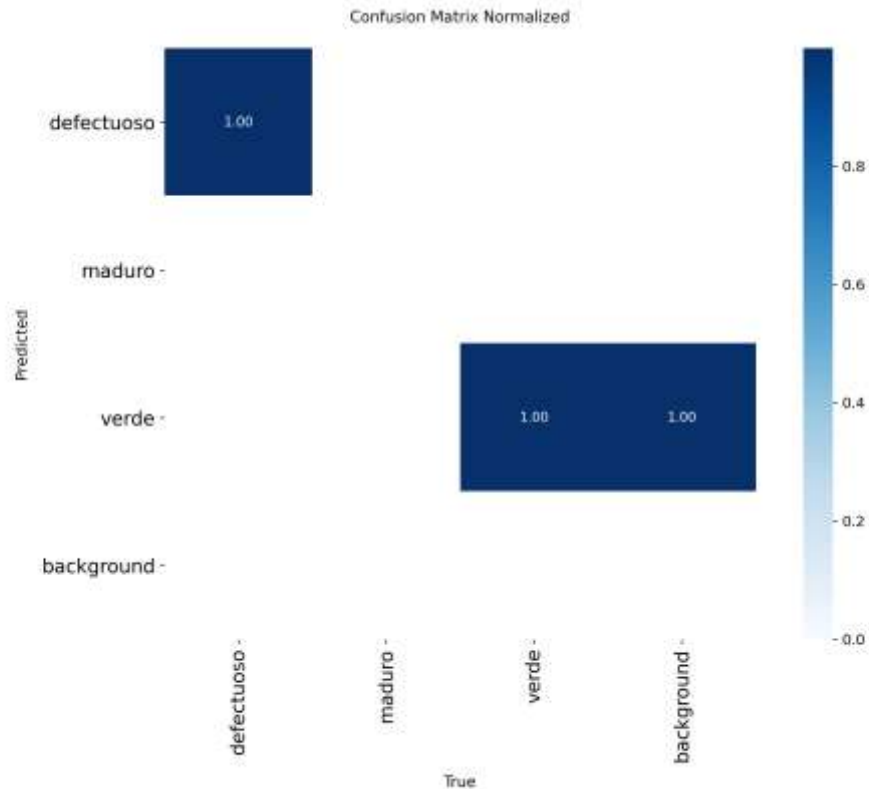


Imagen 5.3.3.2 matriz de confusion normalizada

5.3.4 Resultados visuales

Los resultados visuales obtenidos durante el entrenamiento del modelo permiten observar de manera práctica su desempeño en la detección y clasificación de tomates. En las imágenes presentadas, el sistema identifica correctamente los objetos mediante el uso de cajas delimitadoras (bounding boxes), acompañadas de etiquetas que indican la clase correspondiente, como maduro, verde o defectuoso. Se puede apreciar que el modelo es capaz de reconocer tomates en distintas condiciones, incluyendo variaciones de iluminación, posición y tamaño, lo que evidencia una adecuada capacidad de generalización. Asimismo, el modelo logra diferenciar entre clases con características similares, proporcionando predicciones consistentes y confiables. Estos resultados confirman que el modelo no solo presenta buen rendimiento en las métricas teóricas, sino que también funciona de manera efectiva en escenarios visuales reales.

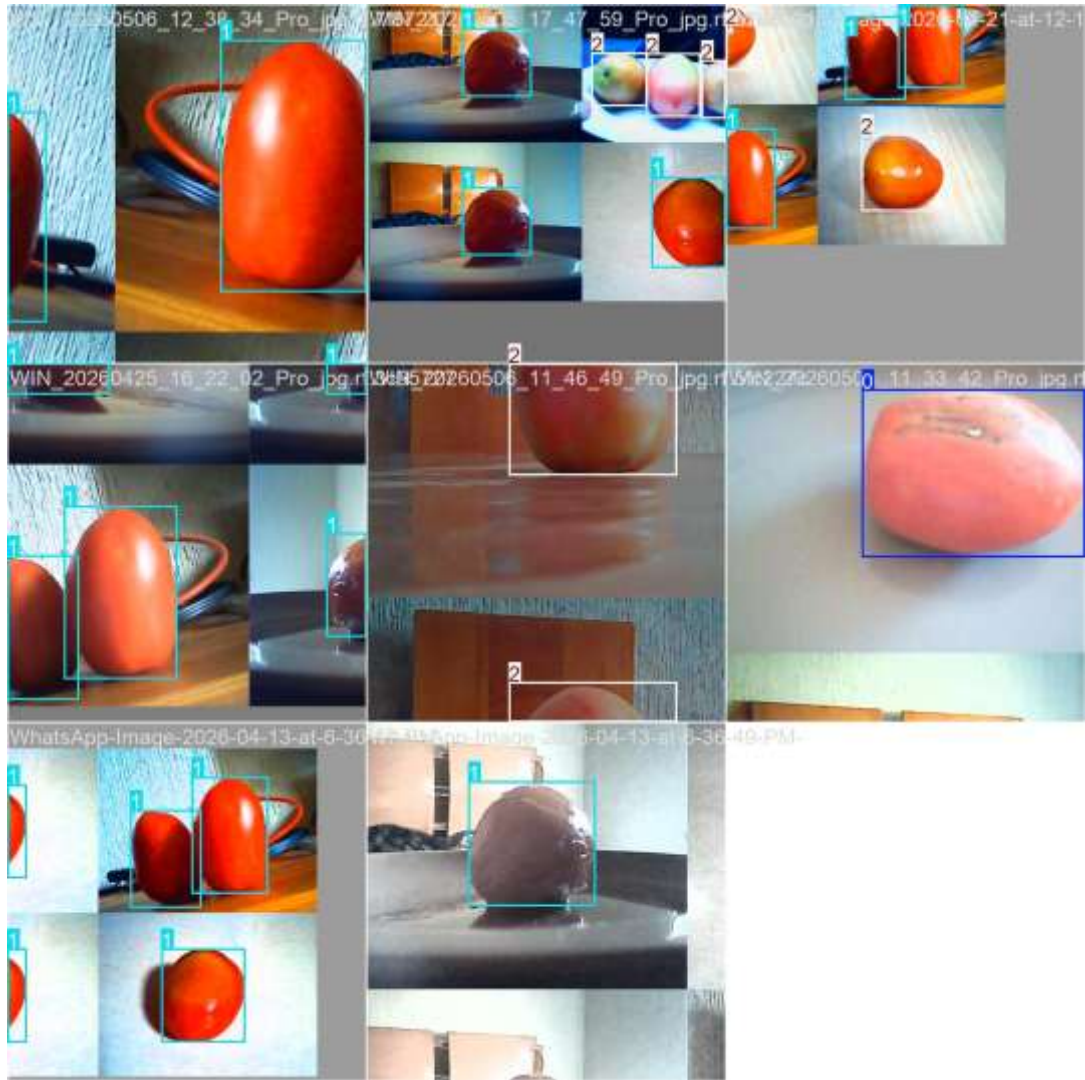


Imagen 5.3.4.1 train batch 0

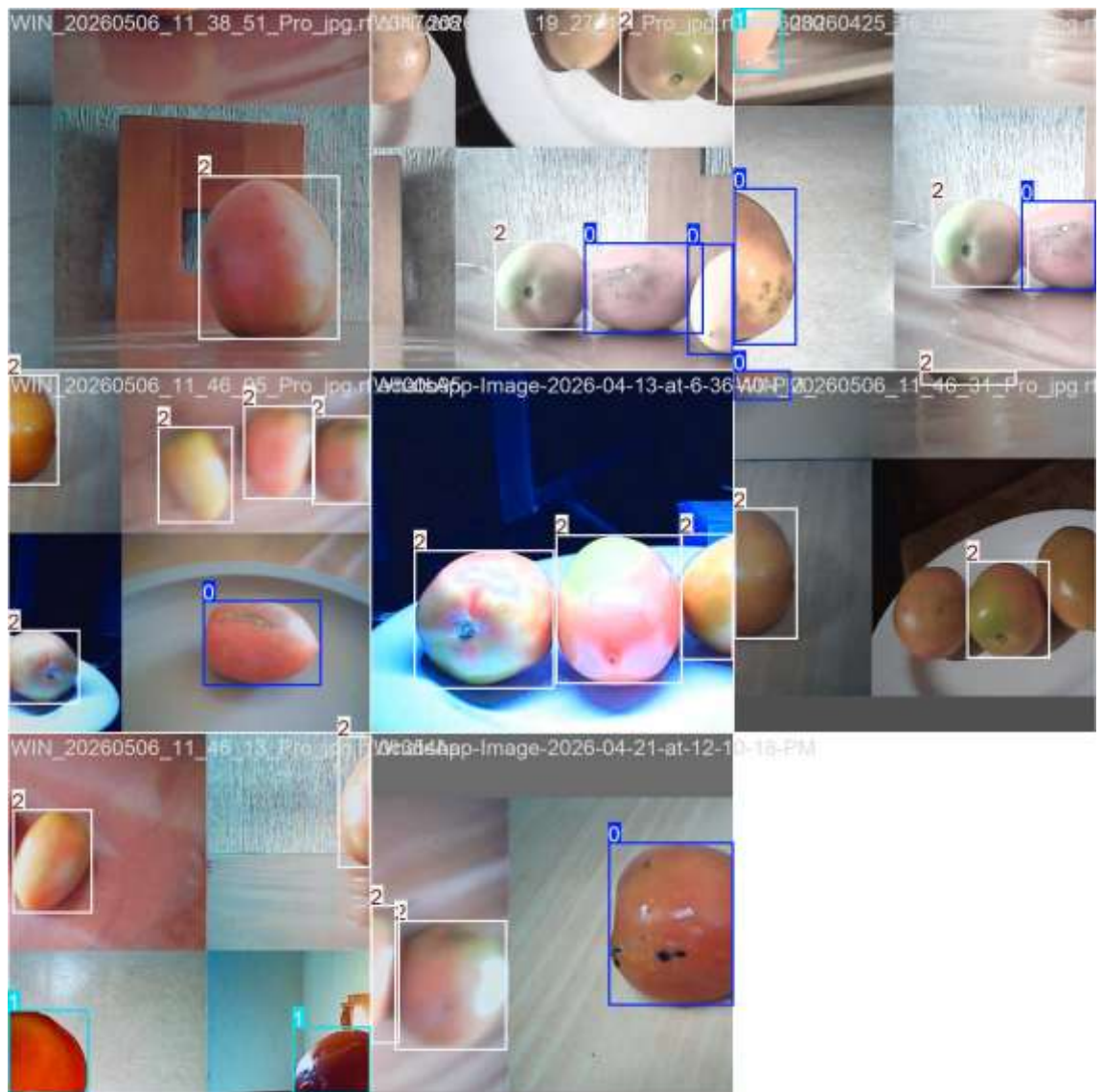


Imagen 5.3.4.2 train batch 1



Imagen 5.3.4.2 train batch 2



Imagen 5.3.4.2 train batch 1440



Imagen 5.3.4.2 train batch 1440

6. Tracking de Objetos (SORT)

El sistema AgroVision incorpora un módulo de seguimiento de objetos basado en el algoritmo SORT (Simple Online and Realtime Tracking), el cual permite mantener la identificación de los tomates detectados a lo largo del tiempo. A diferencia del modelo de detección, que analiza cada frame de manera independiente, el tracking se encarga de asociar las detecciones consecutivas para dar continuidad a cada objeto.

El funcionamiento de SORT se basa en técnicas de estimación de movimiento, específicamente mediante el uso de un filtro de Kalman, el cual predice la posición futura de los objetos en el siguiente frame. Posteriormente, estas predicciones se comparan con las detecciones actuales utilizando una métrica de similitud conocida como IoU (Intersection over Union), permitiendo asociar correctamente cada objeto detectado con su historial previo.

Gracias a este proceso, el sistema es capaz de asignar un identificador único (ID) a cada tomate detectado, el cual se mantiene mientras el objeto permanezca visible en la escena. Esto resulta fundamental para evitar conteos duplicados y para realizar un seguimiento preciso de cada elemento dentro del video.

En AgroVision, el tracking se integra directamente con el modelo de detección, permitiendo no solo ubicar los tomates, sino también asociar su estado (maduro, verde o defectuoso) a cada ID. De esta manera, se garantiza que la información almacenada y mostrada en el dashboard sea consistente a lo largo del tiempo.

En conjunto, el uso de SORT permite mejorar significativamente la robustez del sistema, ya que añade una dimensión temporal al análisis, facilitando el monitoreo continuo de los objetos y proporcionando resultados más estables y confiables en aplicaciones de visión por computadora en tiempo real.

7. Evaluación del Modelo

El apartado de evaluación del modelo se llevó a cabo mediante el uso del método `model.val()`, el cual permite obtener métricas finales que reflejan el desempeño real del modelo sobre datos de validación. En este proceso se calcularon indicadores clave como el mAP evaluado en 0.5, que mide la capacidad del modelo para detectar correctamente los objetos, y el mAP evaluado 0.5:0.95, que representa una evaluación más estricta considerando distintos niveles de precisión en la localización.

Asimismo, se obtuvieron las métricas de precisión (`precision`) y `recall`, las cuales permiten analizar el balance entre la exactitud de las detecciones y la capacidad del modelo para identificar todos los objetos presentes. Los resultados obtenidos muestran valores elevados de mAP, lo que indica una alta capacidad de detección, mientras que los valores de precisión y `recall` reflejan un desempeño adecuado con algunas ligeras inconsistencias propias de la complejidad del problema. En conjunto, estas métricas confirman que el modelo entrenado es confiable y adecuado para su aplicación en el sistema AgroVision.

```
ultralytics 8.4.31 Python-3.14.0 torch-2.11.0+cpu CPU (12th Gen Intel Core i5-12450H)
model summary (fused): 71 layers, 3,006,233 parameters, 0 gradients, 8.1 GFLOPs
oc[34msc[1mval: oc[0mFast image access (ping: 0.20.3 ms, read: 36.95.7 MB/s, size: 28.3 KB)

oc[Ksc[34msc[1mval: oc[0mScanning C:\Users\RVega\Documents\01_PROGRAMACION\Python\AgroVisionProyecto\tomates_yolo.v5i.yolov8\valid\labels.cache... 8
Images, 0 backgrounds, 0 corrupt: 100% ————— 8/8 1.2Mit/s 0.0s

oc[K
Class      Images  Instances  Box(P  R      mAP50  mAP50-95): 100% ————— 1/1 1.5s/it 1.5s
all         8         8      0.786  0.873  0.995  0.962
defectuoso  4         4      0.572  1      0.995  0.929
verde       4         4      0.745  0.995  0.995  0.995
Speed: 3.7ms preprocess, 160.4ms inference, 0.0ms loss, 4.1ms postprocess per image
Results saved to oc[1mC:\Users\RVega\Documents\01_PROGRAMACION\Python\AgroVisionProyecto\runs\detect\val0m
METRICAS GUARDADAS:
{'mAP50': 0.995, 'mAP50_95': 0.9617881165919284, 'precision': 0.7859709879270145, 'recall': 0.8727329344071107}
```

Imagen 7.1 código para evaluar

Métrica	Valor obtenido	Descripción	Interpretación
mAP 0.5	0.995	Mide la precisión del modelo al detectar objetos considerando un umbral IoU de 0.5	Excelente desempeño, el modelo detecta casi todos los objetos correctamente
mAP 0.5:0.95	0.9617	Métrica más estricta que evalúa la detección en diferentes niveles de IoU	Muy buen rendimiento general, incluso bajo condiciones más exigentes
Precision	0.7859	Indica qué tan correctas son las predicciones realizadas	Buen nivel, aunque existen algunos falsos positivos
Recall	0.8727	Mide la capacidad del modelo para detectar todos los objetos reales	Alto rendimiento, el modelo detecta la mayoría de los tomates

Tabla 7.2 Metricas de evaluacion

8. Implementación del Sistema

La implementación del sistema AgroVision se llevó a cabo mediante una arquitectura modular en lenguaje Python, lo que permitió integrar de manera eficiente los distintos componentes del proyecto, incluyendo la captura de video, la detección de objetos, el seguimiento y la visualización de resultados. Cada módulo fue desarrollado de forma independiente, facilitando su mantenimiento, escalabilidad y comprensión dentro del sistema completo.

El módulo principal, contenido en el archivo main.py, se encarga de coordinar el flujo general del sistema, gestionando la captura de imágenes desde la cámara, la ejecución del modelo de detección YOLOv8 y la actualización del tracking mediante el algoritmo SORT. Este flujo se ejecuta en un bucle continuo que permite procesar frames en tiempo real, garantizando una respuesta rápida y constante. Para la detección de objetos, se implementó el módulo detector.py, el cual carga el modelo previamente entrenado y realiza inferencias sobre cada frame. Este módulo devuelve las coordenadas de las cajas delimitadoras, la clase detectada y el nivel de confianza, información que posteriormente es utilizada por el sistema de seguimiento y visualización.

El seguimiento de objetos se implementa en el módulo `tracker.py`, utilizando el algoritmo SORT para asignar un identificador único a cada detección. Esto permite mantener la consistencia de los objetos a lo largo del tiempo y evita duplicaciones en el conteo, mejorando la precisión del sistema en aplicaciones prácticas. Por otro lado, el módulo `camera.py` se encarga de la captura de imágenes en tiempo real desde la cámara del dispositivo, optimizando parámetros como resolución y velocidad de captura. Asimismo, el sistema incorpora un módulo de visualización mediante OpenCV, el cual muestra los resultados directamente en pantalla, incluyendo bounding boxes, etiquetas de clase y métricas como los FPS.

Finalmente, la implementación incluye un dashboard web desarrollado con Flask, contenido en el módulo `dashboard.py`, que permite visualizar de manera remota la información procesada por el sistema. Este dashboard se actualiza en tiempo real a través de una API interna, mostrando el conteo de tomates y su clasificación, lo que proporciona una interfaz amigable para el usuario. En conjunto, la implementación del sistema demuestra la correcta integración de múltiples tecnologías de visión por computadora, procesamiento en tiempo real y desarrollo web, dando como resultado una solución funcional, eficiente y adaptable a distintos escenarios de aplicación.



Imagen 8.1 implementacion

9. Resultados del Sistema

El sistema AgroVision desarrollado cumple con la funcionalidad principal de detección, clasificación y seguimiento de jitomates en tiempo real mediante el uso de visión artificial y modelos de aprendizaje profundo (YOLOv8).

Durante las pruebas realizadas, el sistema fue capaz de identificar correctamente jitomates dentro del campo visual de la cámara, clasificándolos en tres categorías principales: maduro, verde y defectuoso. Además, se implementó un mecanismo de seguimiento (tracking) que permite asignar un identificador único a cada objeto detectado, evitando confusiones entre diferentes instancias a lo largo del tiempo.

En la siguiente imagen se muestra un ejemplo de funcionamiento del sistema:

En esta captura se pueden observar los siguientes elementos:

- Un jitomate detectado en primer plano.
- Un cuadro delimitador (bounding box) que encierra el objeto identificado.
- La etiqueta del modelo que indica:
 - ID del objeto (ID 9).
 - Clase detectada ("verde").
 - Nivel de confianza (0.71).
- Un indicador de rendimiento del sistema en tiempo real (FPS $\approx$ 6), lo que refleja la velocidad de procesamiento de la aplicación.

Los resultados obtenidos muestran que el sistema es capaz de:

- Detectar objetos correctamente incluso con variaciones de iluminación y fondo.
- Clasificar con precisión el estado del jitomate basándose en sus características visuales.
- Mantener el seguimiento del objeto a través de múltiples frames sin perder su identidad.
- Operar en tiempo real, aunque con una tasa de cuadros por segundo moderada debido al uso de CPU.

Por otro lado, durante las pruebas también se observaron algunas limitaciones del sistema:

- La velocidad de procesamiento (FPS) puede disminuir dependiendo de la capacidad del hardware.
- La precisión de clasificación puede verse afectada en condiciones de iluminación extrema o cuando hay similitud visual entre clases.
- El sistema puede generar pequeñas variaciones en la confianza dependiendo del ángulo o posición del objeto.

En general, el sistema AgroVision demuestra un desempeño funcional y confiable para la detección y clasificación de jitomates, cumpliendo con los objetivos planteados del proyecto y proporcionando una base sólida para aplicaciones futuras como monitoreo agrícola automatizado o sistemas de clasificación en línea de producción.

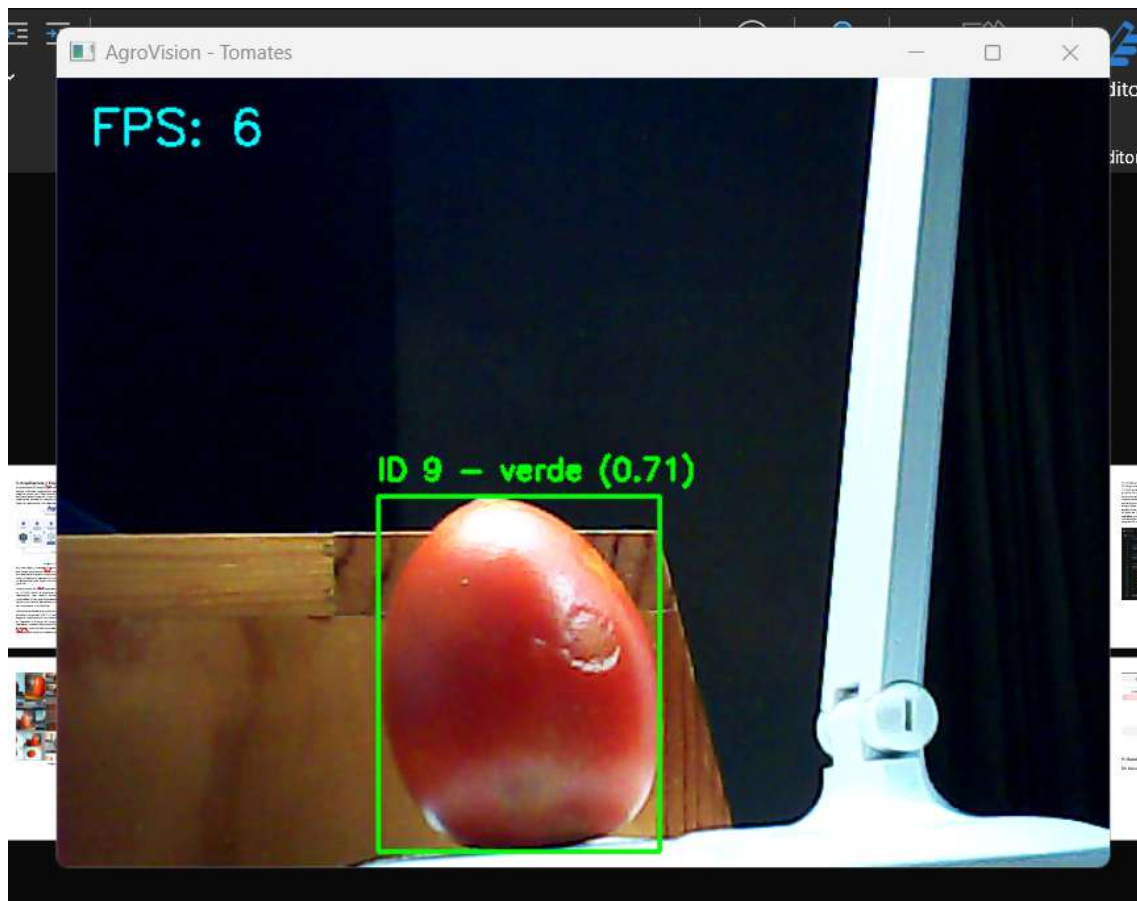


Imagen 10.1 aplicación corriendo

11. Conclusiones

El desarrollo del sistema AgroVision para la detección y clasificación de jitomates permitió demostrar la viabilidad de aplicar técnicas de visión artificial y aprendizaje profundo en soluciones prácticas orientadas al sector agrícola.

A partir de los resultados obtenidos, se concluye que el modelo implementado basado en YOLOv8 es capaz de realizar detección de objetos en tiempo real con un nivel de precisión adecuado para aplicaciones de monitoreo y clasificación básica. El sistema logra identificar jitomates en el entorno capturado por la cámara, clasificándolos correctamente en categorías como maduro, verde y defectuoso, lo que cumple con el objetivo principal del proyecto.

Asimismo, la incorporación de un sistema de seguimiento (tracking) permite mantener la identidad de los objetos detectados a lo largo del tiempo, lo que aporta un valor adicional al sistema al facilitar el conteo y análisis continuo de los frutos. Esto constituye un avance relevante respecto a sistemas que únicamente realizan detección por frame sin correlación temporal.

En términos de desempeño, el sistema opera en tiempo real, aunque con una tasa de cuadros por segundo limitada cuando se utiliza procesamiento por CPU. Esto indica que el rendimiento podría mejorar significativamente si se implementa en hardware con aceleración gráfica (GPU), lo cual abre posibilidades de optimización futura.

Por otro lado, durante las pruebas se identificaron ciertas limitaciones. Entre ellas destacan la sensibilidad del modelo a condiciones de iluminación, ángulos de captura y similitudes visuales entre clases, lo que puede afectar la precisión en algunos casos. Además, el conteo basado en detecciones continuas puede generar repeticiones si no se implementan estrategias más avanzadas de filtrado o lógica de eventos.

A pesar de estas limitaciones, el sistema cumple satisfactoriamente con los requerimientos planteados, ofreciendo una solución funcional, escalable y aplicable en diferentes contextos. El proyecto sienta las bases para futuras mejoras, tales como:

- Implementación de conteo más preciso basado en eventos (entrada/salida de objetos).
- Mejora del modelo mediante mayor cantidad y diversidad de datos de entrenamiento.
- Optimización del rendimiento mediante el uso de GPU u otros dispositivos aceleradores.
- Integración en sistemas más complejos de monitoreo agrícola o clasificación automatizada.

En conclusión, AgroVision constituye una herramienta eficaz para la detección y clasificación de jitomates en tiempo real, demostrando el potencial de la inteligencia artificial como apoyo en procesos agrícolas y de automatización, y evidenciando que el enfoque implementado es adecuado para evolucionar hacia soluciones más robustas en entornos productivos.

12. Referencias

- González, R. C., & Woods, R. E. (2008). Procesamiento digital de imágenes. Pearson Educación. Recuperado de: <https://archive.org/details/ProcesamientoDeImagenesDigitales2daEdicionRafaelC.GonzalezRichardE.Woods/page/n3/mode/2up> [archive.org]
- Ultralytics. (2023). Documentación de YOLOv8. Recuperado de: <https://docs.ultralytics.com/es/models/yolov8/> [docs.ultralytics.com]
- Domínguez Mínguez, T. (2025). Visión artificial: aplicaciones prácticas con OpenCV-Python. Recuperado de: <https://www.perlego.com/es/book/2702842/visin-artificial-aplicaciones-prcticas-con-opencv-python-pdf> [perlego.com]
- Jouanne, C. (2023). OpenCV-Python: Tutorial en español. Recuperado de: <https://cjjouanne.github.io/OpenCV-Python/> [cjjouanne.github.io]
- Russo, C., et al. (2018). Visión artificial aplicada en agricultura de precisión. Recuperado de: https://sedici.unlp.edu.ar/bitstream/handle/10915/68308/Documento_completo.pdf-PDFA.pdf?sequence=1 [sedici.unlp.edu.ar]